Baker College | GREATNESS IS EARNED HERE.

Micro-Level PLC Programming

Programmable Logic Controllers

Brad Peirson
College of Information Technology and Engineering

Agenda

Architectural Overview

Micro-Level Programming

Student Learning Outcomes

1. Create programs in Ladder Logic using the following instructions in an application
2. Program PLC timers and counters

Architectural Overview

The Cost of Spaghetti Logic

$260,000

Average Hourly Downtime Cost

- High-throughput automotive: $2.3M per hour
- Unstructured logic *directly* prolongs diagnosis time

Bad Logic Compounds Downtime

- Extended Mean Time to Repair (MTR): Up to 80% of downtime is spent deciphering code rather than fixing hardware
- Troubleshooting Friction: Intermingled safety permissives, sequence logic, and direct I/O forces trial-and-error debugging
- Modification Risk: Fixing one rung accidentally breaks secondary machine routines

Spaghetti Code vs. Architecture

Unstructured Logic

Direct physical I/O scattered across dozens of subroutines with unbuffered signals and nested latches

Layered Architecture

Clean separation between Input Mapping, State Machines, Output Buffers, and Alarm Routines.

Pillars of Industrial PLC Code

1. Readability

Code must be self-documenting for field technicians at 2 AM. Clear tag naming conventions and modular layout eliminate guesswork during emergency outages.

2. Maintainability

Hardware shifts and I/O module reconfigurations shouldn't break core sequence logic. I/O abstraction buffers protect the software core from physical changes.

3. Standard Troubleshooting

Standardized step sequencing and diagnostic alarms pinpoint stalled steps immediately, eliminating the need to search through hundreds of rungs.

Micro-Level vs. Macro-Level

- High-quality industrial code requires mastery at both the instruction level and architectural level

Micro (Rung Mechanics) How individual instructions, logic gates, and execution operate within a single scan cycle

Macro (Program Layout) How routines, modules, are organized across the entire controller

- Flawless micro-logic fails inside a disorganized architecture; brilliant macro-architecture is useless if individual rungs are buggy

Micro-Level (Rung Mechanics)

Boolean Logic & Flow Optimizing examine if closed (`XIC`), examine if open (`XIO`), and utput coils (`OTE`, `OTL`, `OTU`)

Scan Cycle Dynamics Understand top Understanding top-to-bottom, left-to-right rung evaluation and memory image updates

Edge Detection & Timing Proper use of One-Shots (ONS/OSR) and timers (TON/TOF) to prevent race conditions

Micro-Level Best Practices

- Keep rungs readable; avoid excessively nested logic
- Limit multiple outputs driven by identical conditions across different rungs (destructive double-coils)

Macro-Level (Program Layout)

Modular Organization Dividing code into Tasks (Periodic vs. Continuous), Programs, and Subroutines/Routines

State Machine Frameworks Structuring sequences (e.g., PackML, ISA-88, or custom Step/State logic)

Data Structuring Utilizing User-Defined Data Types (UDTs) and arrays over unorganized tags

Macro-Level Best Practices

- Enforce a standardized folder/routine hierarchy (e.g., `00_Inputs`, `10_Modes`, `20_Sequence`, `90_Outputs`)
- Decouple hardware I/O mapping from core process control logic

Micro vs. Macro Side-by-Side

Aspect	Micro-Level (Rung Mechanics)	Macro-Level (Program Layout)
Primary Scope	Single line of logic / localized operation	System-wide architecture / overall code base
Main Objective	Deterministic evaluation & correct bit manipulation	Scalability, maintainability & clear fault isolation
Typical Tools	Timers, counters, logic gates, math operations	Subroutines, UDTs, State Machines, Task Scheduling
Failure Mode	Intermittent bugs, missed pulses, logic race conditions	Unmaintainable “spaghetti code,” difficult commissioning

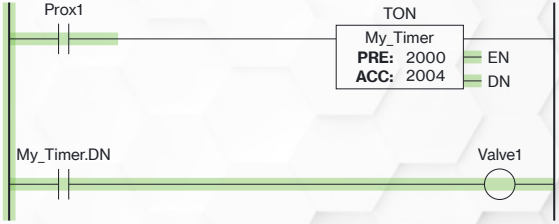
Micro-Level Programming

Micro-Level Programming Rules

- Micro-level (rung level) programming is governed by a series of fixed rules
- Some of the rules are based on the idea that ladder logic is based on electrical schematics
- Some software (Logix Designer) will allow breaking the rules–it's up to the *programmer* to follow them

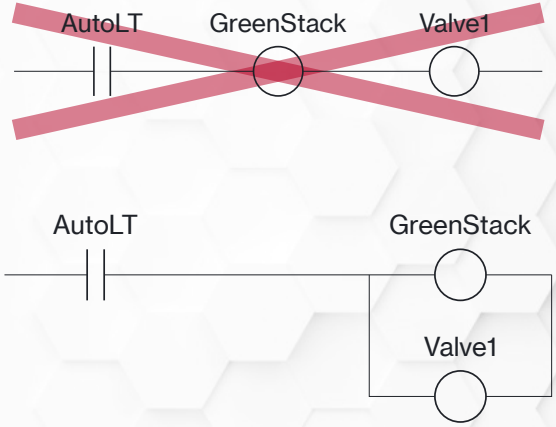
Rule 1 - Rung Construction

- Each program line is a *rung*
- *Condition* instructions go to the left
- *Control* instructions go to the right
- Each rung must contain *at least* one control
- Most rungs contain condition *and* control



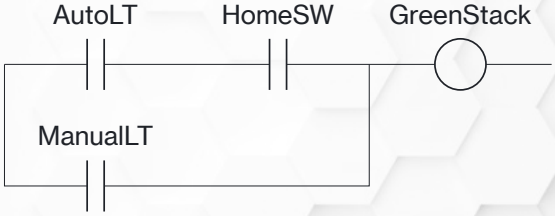
Rule 2 - One Control per Branch

- Each branch (level) of a rung can only have *one* control instruction
- Control instructions can be placed in parallel but *never* series



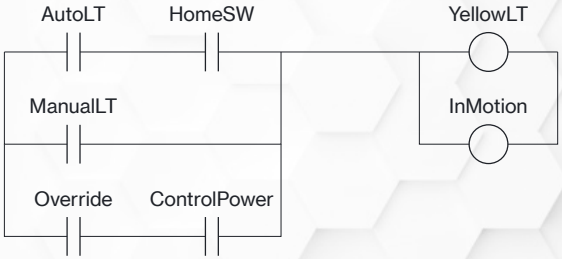
Rule 3 - Condition Combinations

- Condition instructions can be in series, parallel, or any combination



Rule 4 - Branching

- Conditions *and* controls can be placed in branches
- Branches can be *nested*–have different start and end points
- Branches *cannot* overlap

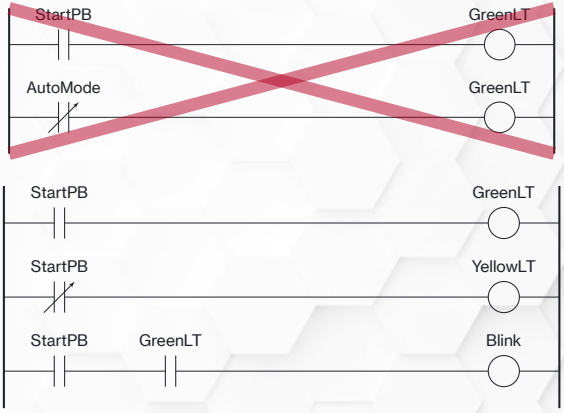


Rule 5 - Condition Variables

- Variables and I/O can be used in multiple condition instructions

Rule 6 - Control Variables

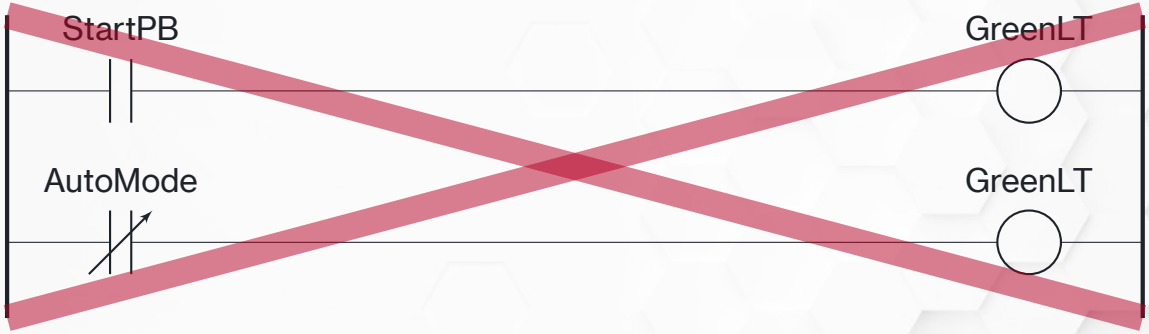
- Variables and I/O can only be used in *one* control instruction (`OTE`)
- Those variables *CAN* be used in other condition instructions



The Single OTE Trap (1 of 3)

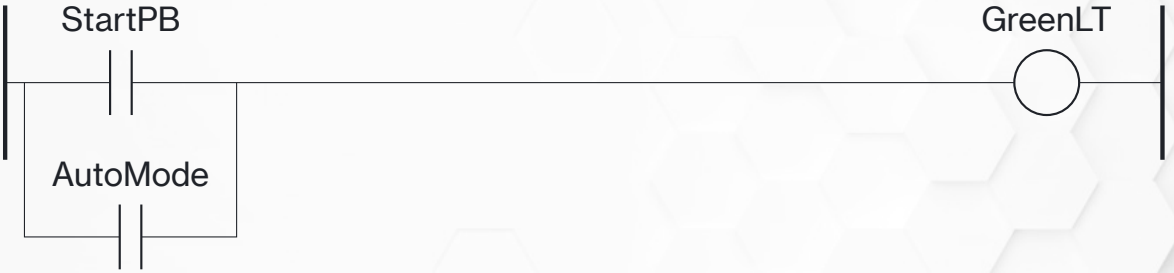
What happens to the output variable in this program?

The Single OTE Trap (2 of 3)



Remember the scan–the first rung is scanned, the output is true. The output will *never turn on*. The logic scan *stores* the value, and it's going to store whatever value it hit last.

The Single `OTE` Trap (3 of 3)



There are a *lot* of cases where you *want* to use an `OTE` multiple times on the same variable–this is what branching is for

Micro-Level Programming

I/O Mapping

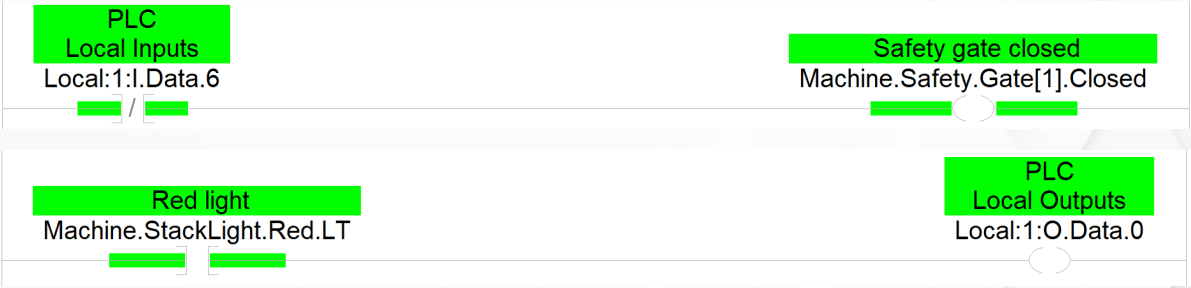
Direct I/O Aliasing

- Direct aliasing assigns a physical I/O point to an internal tag
- This is *not recommended*
- I/O are *physical devices*–physical devices occasionally break
- The system must be taken offline to change the alias and move the tag to another I/O point

I/O Mapping

- *Mapping* is the preferred method of assigning I/O points to tags
- Two subroutines are created–one for input mapping, one for output mapping
- One I/O point per rung
- A simple `XIC` and `OTE` combination is used to assign the I/O to the tag

I/O Mapping Example



Top: when the physical input `Local:1:I.Data.6` turns on, it will turn on the tag `Machine.Safety.Gate[1].Closed`. Bottom: when the tag `Machine.StackLight.Red.LT` turns on, it will turn on the physical output `Local:1:O:Data.0`.

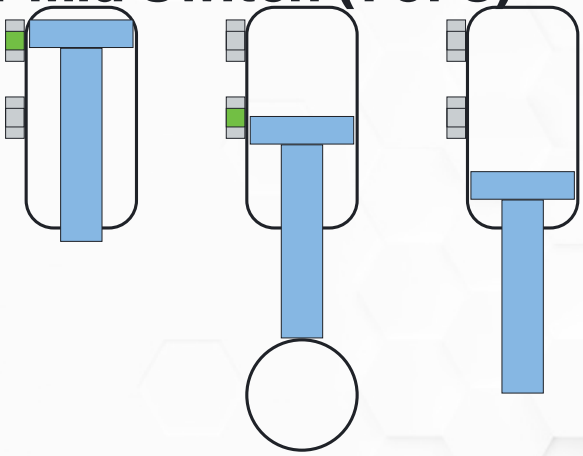
Micro-Level Programming

Input Processing

Input Debouncing

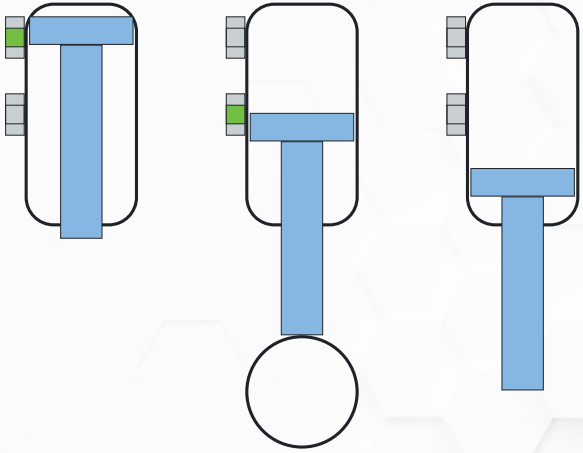
- Mapping also allows something aliasing does not–*debouncing*
- Debouncing allows us to intentionally delay the time between a physical input turning on and the tag it's tied to turning on
- Not required for *all* input points
- Good for filtering out "nuisance" trips caused by noise
- Good for any input that has a tendency to "blink"–allows us to be *sure* it's on before we use it

Air Cylinder Mid Switch (1 of 3)



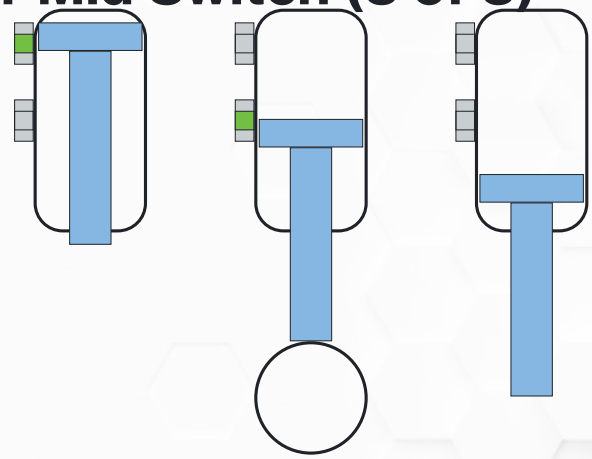
This is a case-study, it actually happened: the air cylinder is acting as a clamp. The “extend” switch is *not* at the end of the air cylinder, it’s in the middle set where the cylinder would stop if a part were present.

Air Cylinder Mid Switch (2 of 3)



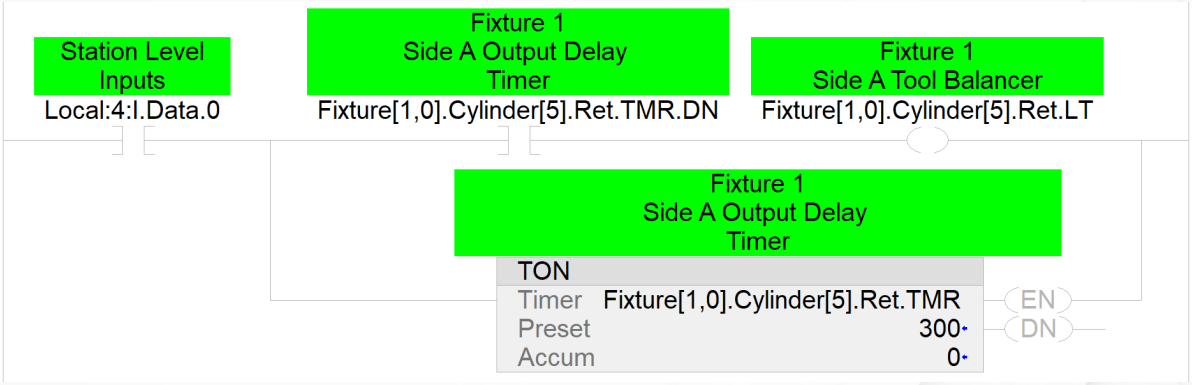
This setup is also intended to act as a part-present sensor–if we extend the cylinder and the switch doesn't turn on that means the cylinder extended too far, and there is no part.

Air Cylinder Mid Switch (3 of 3)



When there is no part, the cylinder flies by that switch *really fast*. Except, because we don't know where we are in the scan *sometimes* it catches the "blink" of the switch and counts it as on–the logic thinks there's a part there and the machine carries on.

Debounce Logic



Debounce is done with a single timer. When the physical input turns on, the timer starts. The *tag* doesn't turn on until (unless) the timer `DN` bit turns on. Debounce timers don't need to be very long–50-200 ms is usually plenty.

Analog Scaling

- Analog inputs come in as integers–whole numbers that represent a voltage or current value from the sensor
- Most analog cards are 12-bit–the value received by the PLC will be 0-4096
- You *can* use this value, but it doesn't mean much to anyone else looking at the code
- It is *much* better to scale it to engineering units *before* using it in logic

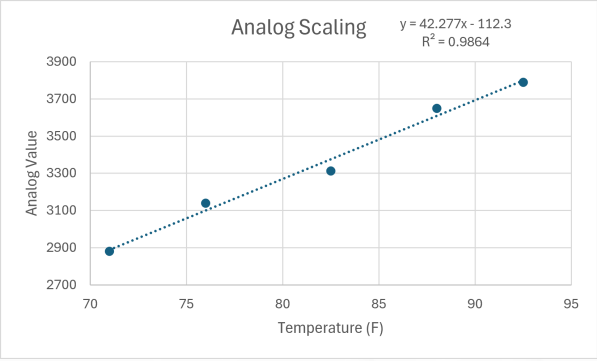
Allen-Bradley Scaling

- Allen-Bradley's preferred method in the Logix 5000 PLCs is to scale the value in the definition of the analog card itself
- This works, but has serious drawbacks
- Not all cards and input devices are equal–if you swap temperature sensors and it's off compared to the old one the scaling needs to be changed
- If the scaling is in the card definition, the system needs to be taken offline to change it
- Plus, not all manufacturers do it this way–we like "hardware agnostic" programming methods

Setting the Scale (1 of 2)

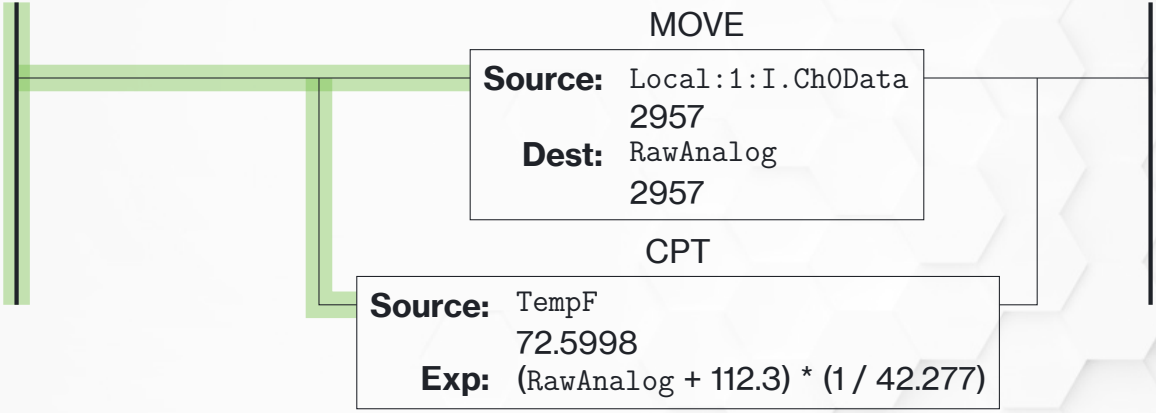
- Math would tell us the scaling factor would be:
  Engineering Units = `UnitRange` × $\frac{\texttt{AnalogValue}}{\texttt{MaxValue}}$
- This only works *if* the signal starts at 0
- It is also only ever going to get you close–not all sensors and cards are equal
- Best practice is to actually *measure* the conversion factor
- In this case, a spreadsheet is your best friend…

Setting the Scale (2 of 2)



Set the sensor to 5-6 known real-world values and read the analog input value. Plot them in a spreadsheet and run a *linear regression*–which the software will do automatically with one click. The regression equation is the scaling factor.

Scaling in the Input Map

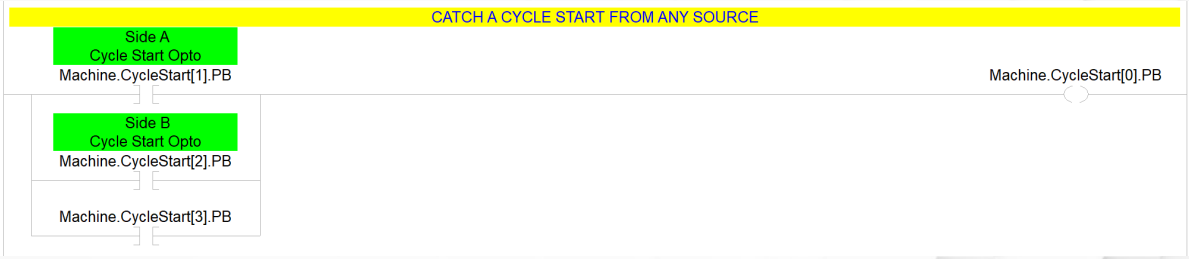


The preferred method is to scale the analog value to actual units *in the input map*. The rest of your program is instantly easier to troubleshoot–you're using degrees throughout the program instead of analog count.

Logical Input Grouping

- Input conditions can get extremely complicated, even in small machines
- An `Inputs` subroutine can be used to group inputs together, making the main machine sequence much simpler to follow
- This is commonly use for home conditions, but can be used to simplify anything

Inputs Subroutine Example 1



Grouping inputs together in a logical manner greatly simplifies the main sequence routine

Inputs Subroutine Example 2

This rung combines all of the inputs that must be true in order for a FANUC robot to accept a start command.

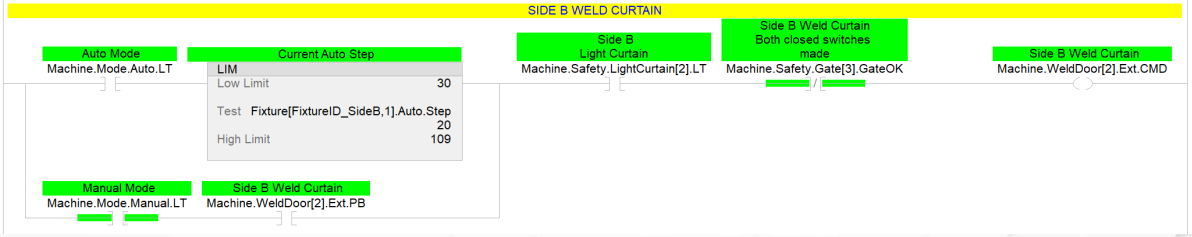
Micro-Level Programming

Output Management

Outputs Subroutine

- Where input grouping is used for readability of the logic, output grouping is *necessary* for machine safety
- An `Outputs` subroutine is used to create the *permissives* for machine motion–those conditions that are required to make sure the machine doesn't crash into itself

Output Permissive

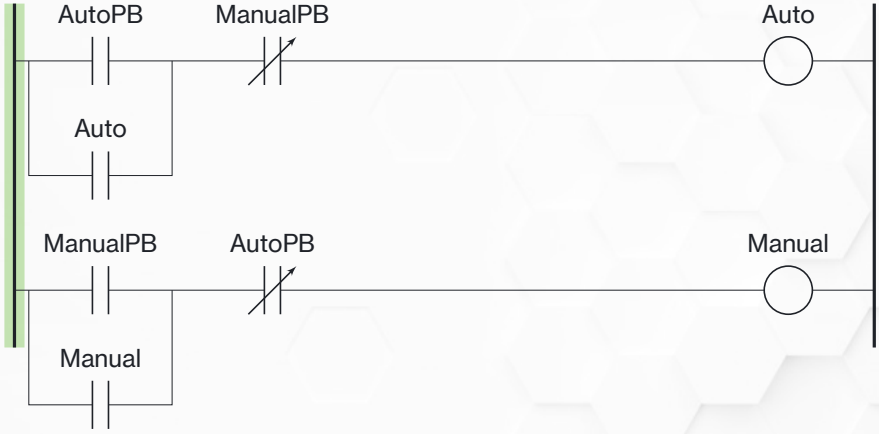


This logic will not allow the `Machine.WeldDoor[2].Ext.CMD` output to energize unless the other elements of the machine are clear

Output Seal-In

- Output latching instructions and retentive outputs can be dangerous–they're still energized when the PLC power cycles
- A *seal-in* is the preferred method
- A seal-in uses only an `OTE` instruction, guaranteeing that the output will turn off on a power cycle

Seal-In Logic

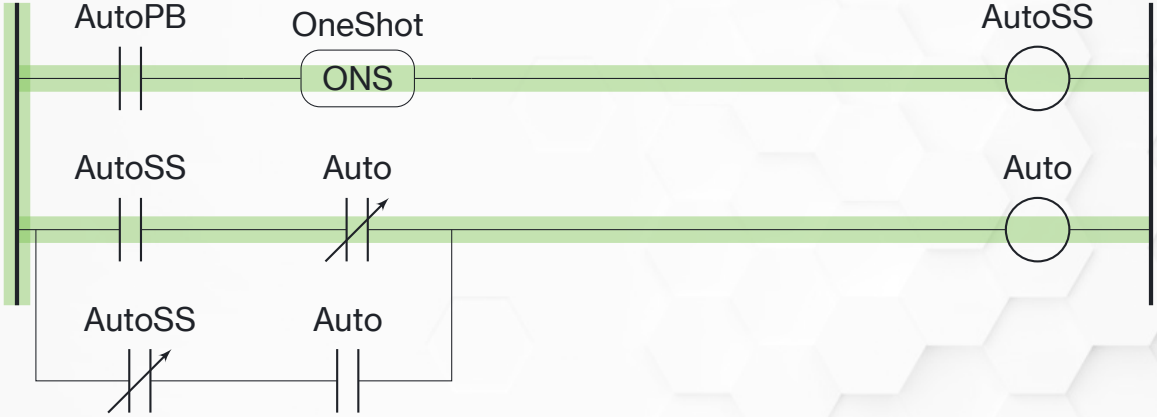


Auto will remain energized until someone presses the manual button, and manual will remain energized until someone presses the auto button. Both will turn off if power is lost.

Momentary as Toggle

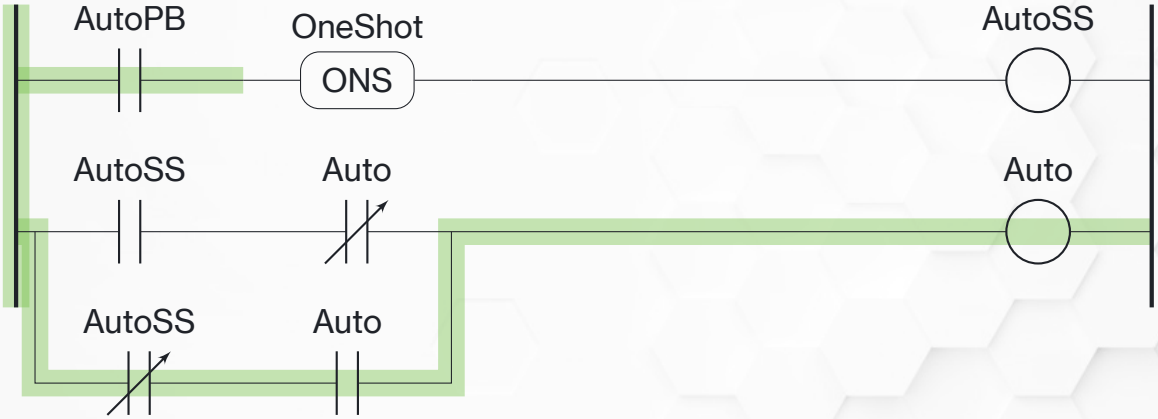
- Buttons are available in momentary and maintained configurations
- HMI configuration software includes the same options
- We can write ladder logic to treat a momentary button as a toggle
- Can be used with hardware buttons, but most useful with HMIs–simplifies development knowing *all* HMI buttons are momentary

Toggle Logic - First Scan



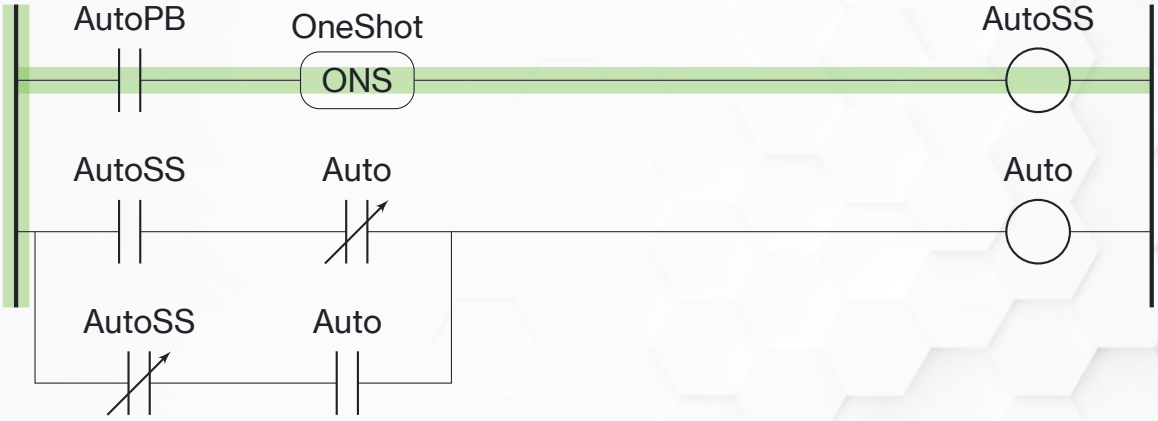
Rung 1: The button is pressed, the oneshot is true, the single scan bit energizes.
Rung 2: The single scan bit is true, but auto is still false so auto will energize.

Toggle Logic - Second Scan



Rung 1: The button is still pressed but the oneshot is now false, the single scan bit turns off.
Rung 2: The single scan bit is false but auto is already energized, so it stays energized.

Toggle Logic - Second Button Press



Rung 1: The button is pressed, the oneshot is true, the single scan bit will energize.
Rung 2: The single scan bit is true, but so is auto–there is no true branch so auto will de-energize.

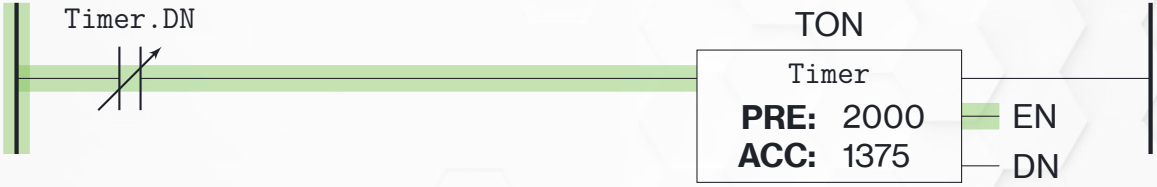
Micro-Level Programming

Free-Running Timers

Free-Running Timers

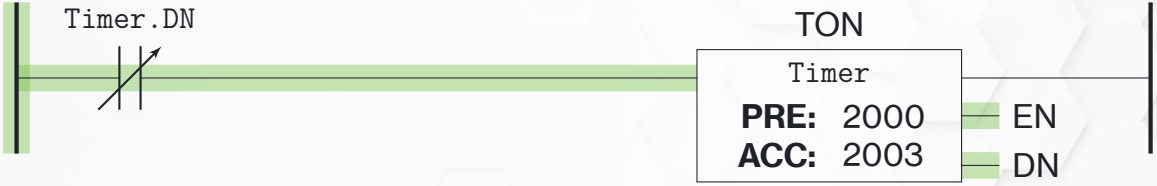
- A free-running timer is simply a timer that resets itself
- The `DN` bit will only be on for one scan–not good for a lot
- But *incredibly useful* for things that need to happen at regular intervals
- For example, log a temperature reading once an hour

Free-Running Timer Logic (1 of 4)



When the PLC first boots the timer is not done, so it will be enabled.

Free-Running Timer Logic (2 of 4)

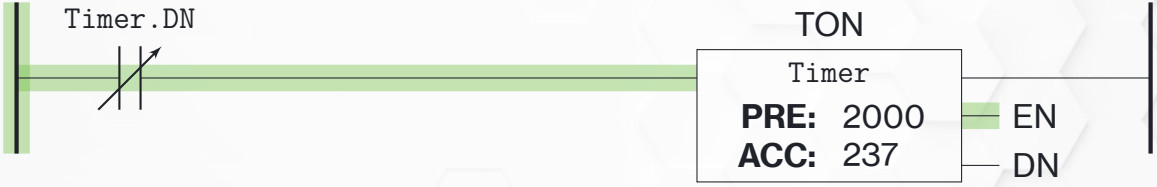


The timer finishes as normal and the `DN` bit energizes.

Free-Running Timer Logic (3 of 4)

The `DN` bit is true for *exactly one scan*–just long enough to reset the timer. It's also long enough for us to use `DN` to trigger other things, like data logging.

Free-Running Timer Logic (4 of 4)



On the very next scan the `DN` bit is false, so the timer starts timiing again.

Micro-Level Programming

Blink Timers

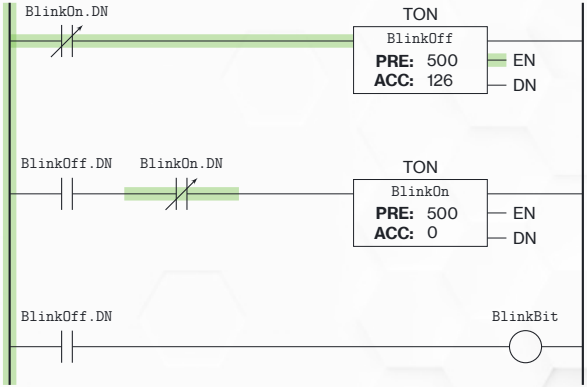
Blink Timers

- Some PLC brands include pre-defined bits that blink at specific intervals
- That would be a hardware specific solution–we want our code to be as universal as possible
- There are several good ways to set use timers to create a blinking bit

Two-Timer Blink

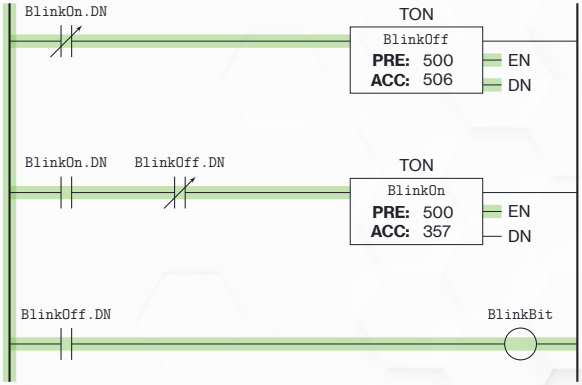
- The two-timer blink method is effectively two free-running timers
- The first timer enables the second
- The second timer resets the first
- The blink happens while the first timer is timing

Two-Timer Blink Logic (1 of 3)



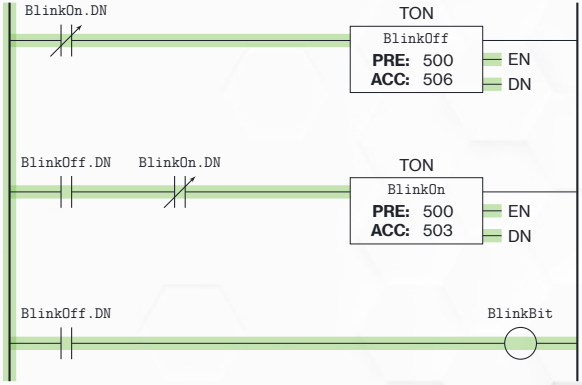
On the first scan the second timer hasn't finished so the first timer starts.

Two-Timer Blink Logic (2 of 3)



The first timer finishes, which enables the second timer and energizes the blink bit.

Two-Timer Blink Logic (3 of 3)



When the second timer finishes its `DN` bit is on for exactly one scan–just long enough to reset both timers.

Micro-Level Programming

Countdown Timers

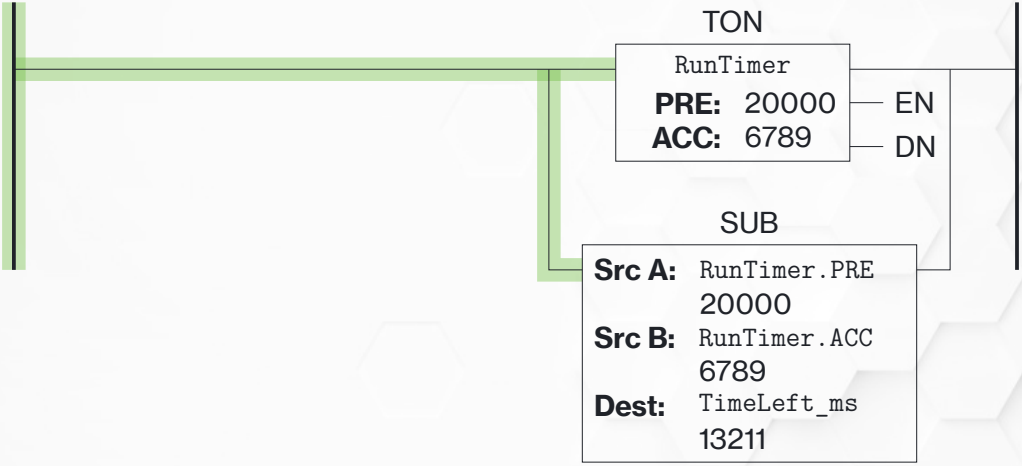
Countdown Timers

- The techniques discussed up to this point are intended primarily to make it easier for technicians and engineers to troubleshoot the code
- Knowing how to properly code a countdown timer is for the *operators*
- The countdown can be displayed on a human-machine interface to let the operator know *exactly* how much time is left in a given operation

How Would You Program a Countdown?

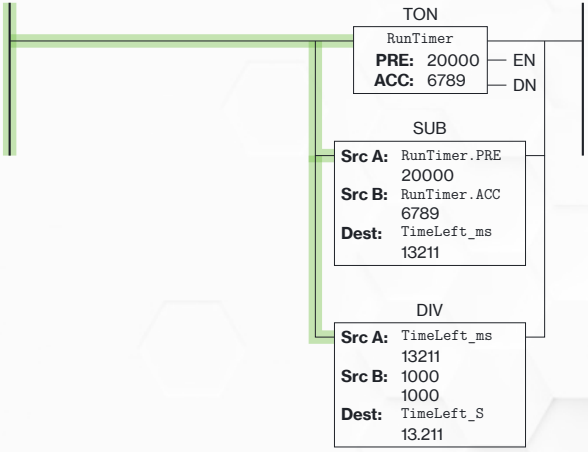
How would *you* program a timer so that the *time remaining* can be displayed to an operator?

Countdown Timer Logic (1 of 2)



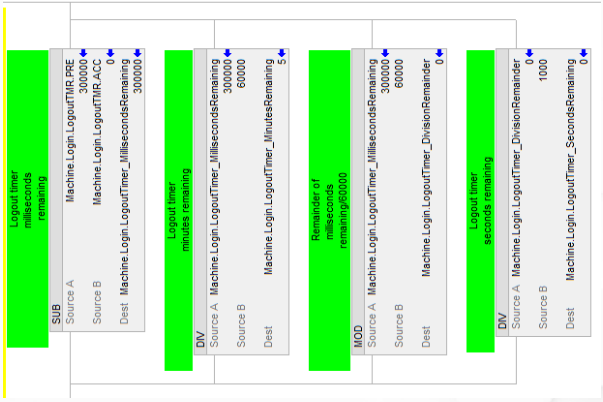
The timer operates as normal and is used elsewhere in the logic just like any timer. The `TimeLeft_ms` will be counting down to zero as the timer runs.

Countdown Timer Logic (2 of 2)



Remember, the timer `PRE` and `ACC` are in *milliseconds*–that doesn't mean much to an operator.

Countdown Timer In Use



This countdown shows an operator how long they have before they are logged out. It displays in minutes *and* seconds.

Micro-Level Programming

The Most Important Micro Concept

Baker College | GREATNESS IS EARNED HERE.

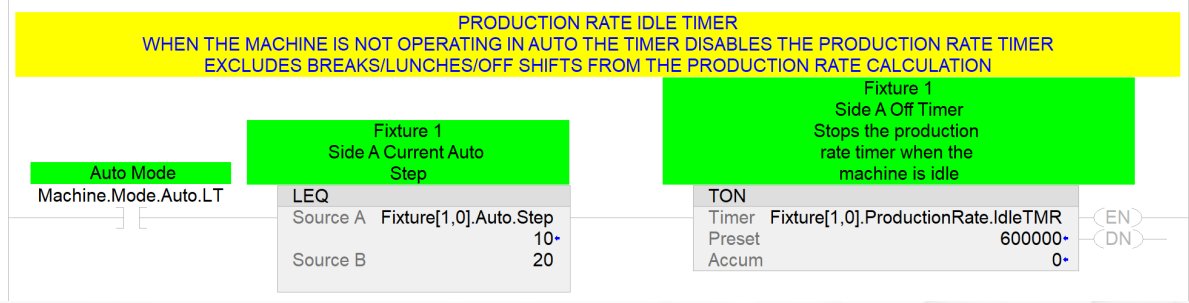
The Most Important Micro Concept

This presentation includes *many* screenshots from an actual, functional machine program–what is present in *all* of them?

Logic Documentation

- All PLC platforms include a way to *document* the code–usually called *commenting*
- Most platforms allow for rung comments *and* tag comments (descriptions)
- **USE THEM BOTH!**
- The benefits are two-fold: a technician troubleshooting at 2am will have an easier time understanding what the program is doing, and in 5 years *you* will remember what you did!

Using Comments



Comments *greatly* simplify troubleshooting–use them everywhere you can.

Questions?

Copyright Notice